Fundamentos de JavaScript y Node.js para desarrollo web

Breve descripción:

JavaScript es el lenguaje que impulsa la web moderna y Node.js su extensión al servidor, juntos constituyen el ecosistema tecnológico de referencia para construir aplicaciones web en tiempo real. Este componente aborda sus fundamentos, la programación orientada a objetos (POO) y las características de Node.js, orientando al aprendiz en la selección argumentada de herramientas según los requisitos técnicos del software a desarrollar.

Agosto 2026

Tabla de contenido

Introducción 4

1. Fundamentos de JavaScript 7

1.1. Variables en JavaScript 7

1.2. Entorno de desarrollo integrado y configuración inicial 8

1.3 Funciones y closures 10

1.4 Objetos y estructuras de datos 12

1.5 Estructuras de control y condicionales 15

1.6 Ciclos 18

1.7. Declaración de funciones y código limpio 22

2. Programación orientada a objetos (POO) en JavaScript 25

2.1. Clases y prototipos 27

2.2 Nombramiento y organización del código 29

2.3 Encapsulación, herencia y composición 31

3. Node.js para tiempo real 35

3.1 Características y beneficios 35

3.2 Bibliotecas y ecosistema npm 37

3.3 Conceptos de TCP y sockets 40

3.4 Ciclo de vida y event loop 42

3.5 Instalación y verificación del entorno 47

3.6 Gestión de dependencias, seguridad y mantenimiento 48

4. Caracterizar y seleccionar herramientas 52

4.1. Criterios de caracterización 52

4.2 Matriz de decisión para tiempo real 53

Síntesis 56
Glosario 58
Referencias bibliográficas 60
Créditos 61

Introducción

El desarrollo de aplicaciones web modernas exige algo más que escribir código funcional: requiere criterio técnico para seleccionar las herramientas adecuadas, justificar esa elección y anticipar sus consecuencias sobre el rendimiento, la seguridad y la mantenibilidad del producto.

JavaScript y Node.js constituyen hoy el ecosistema de referencia para construir estas soluciones, y dominar sus fundamentos es el punto de partida para actuar con autonomía profesional en proyectos reales.

El recorrido inicia con los conceptos esenciales de JavaScript: variables, funciones, objetos, estructuras de control y ciclos. A partir de esa base, se avanza hacia la programación orientada a objetos (POO), donde se estudian las clases, la organización del código y los pilares de encapsulación, herencia y composición. Finalmente, se aborda Node.js desde su arquitectura interna, el `event loop` y el modelo de entrada/salida (I/O) no bloqueante, hasta su ecosistema de bibliotecas (npm), los conceptos de red (TCP y `sockets`) y la gestión de dependencias.

En cada tema se plantean situaciones reales de desarrollo para que la comprensión no se quede en la teoría, sino que se traduzca en decisiones técnicas argumentadas: comparar alternativas, identificar riesgos y comunicar con claridad las razones detrás de cada elección. Esta capacidad de argumentación es lo que distingue a un desarrollador competente de uno que simplemente ejecuta instrucciones.

Video 1. Construcción de aplicaciones web en tiempo real con Node.js



Enlace de reproducción del video

Transcripción del video: fundamentos de JavaScript y Node.js para desarrollo web
Fundamentos de JavaScript y Node JS para desarrollo web. El desarrollo de aplicaciones web modernas exige algo más que escribir código funcional. Requiere criterio técnico para seleccionar las herramientas adecuadas, justificar esa elección y anticipar sus consecuencias sobre el rendimiento, la seguridad y la mantenibilidad del producto. JavaScript y Node JS constituyen hoy el ecosistema de referencia para construir estas soluciones. A través de situaciones reales de desarrollo se evidencia cómo el desconocimiento de los fundamentos del lenguaje puede generar código frágil, vulnerable y difícil de mantener. Reconocer estas consecuencias favorece una actitud técnica consciente y argumentada frente a cada decisión de desarrollo. De esta manera se profundizan los conceptos esenciales de JavaScript: variables, funciones, objetos, estructuras de control y ciclos.

Transcripción del video: fundamentos de JavaScript y Node.js para desarrollo web
A partir de esa base se avanza hacia la programación orientada a objetos POO, explorando clases, encapsulaciones, herencia y composición como pilares de arquitecturas sólidas. Se aborda Node JS desde su arquitectura interna, el event loop y el modelo de entrada/salida (I/O) no bloqueante, hasta su ecosistema de bibliotecas, el gestor de paquetes de Node (npm), los conceptos de red, protocolo de control de transmisión (TCP) y sockets, y la gestión de dependencias con criterios de seguridad profesional. Este enfoque fortalece la capacidad de comparar alternativas, identificar riesgos y comunicar con claridad las razones detrás de cada elección, consolidando la autonomía profesional necesaria para actuar con competencia en proyectos reales de desarrollo web.

1. Fundamentos de JavaScript

JavaScript ha evolucionado de ser un pequeño lenguaje para animar botones a convertirse en la columna vertebral de aplicaciones de escala mundial. Para dominar Node.js, primero se deben entender las raíces del lenguaje.

1.1. Variables en JavaScript

Una variable en JavaScript es un identificador que permite acceder a un valor durante la ejecución del programa. Sin embargo, quedarse solo con la idea de que es únicamente "una caja donde se guardan cosas" lleva pronto a errores difíciles de rastrear: valores que cambian de forma inesperada, datos que se pierden y comportamientos distintos asociados al uso de palabras clave como var, let y const.

Una forma clara de entender este concepto es pensar en una mudanza. Para trasladar objetos de un lugar a otro, no se llevan sueltos; se utilizan cajas. A cada caja se le coloca una etiqueta ("Libros", "Cables", "Documentos") y luego se guarda el contenido en su interior. En JavaScript, una variable funciona exactamente así: una caja con una etiqueta donde se guarda información para utilizarla más adelante en el programa.

En el pasado, JavaScript utilizaba la palabra **var** para crear estas cajas. Sin embargo, el lenguaje ha evolucionado y hoy se emplean dos términos más precisos: **let** y **const**.

- **let (la caja abierta):** se utiliza cuando el contenido de la variable va a cambiar con el tiempo. Por ejemplo, en el desarrollo de un videojuego, el puntaje del jugador se modifica constantemente:

  Javascript

  let puntaje = 0;

puntaje = 150;

- **const (la caja fuerte):** se emplea cuando se desea almacenar un dato que no debe cambiar. Si el sistema intenta modificarlo, JavaScript emitirá un error:

  Javascript

  const añoDeNacimiento = 1995;

  Buena práctica:

  Usar siempre **const** por defecto. Si durante el desarrollo se identifica que el valor necesita cambiar, se modifica la declaración a **let**. Este hábito previene errores inesperados y hace las aplicaciones más estables.

1.2. Entorno de desarrollo integrado y configuración inicial

Una vez comprendidas las variables como cajas etiquetadas, surge una pregunta lógica: ¿qué tipo de elementos se pueden guardar en su interior? Es aquí donde entran en juego los tipos de datos.

JavaScript destaca por ser un lenguaje flexible; a diferencia de otros más estrictos, no es necesario declarar de antemano el tipo de elemento a guardar, pues el propio lenguaje analiza el contenido y determina automáticamente si se trata de texto, un número u otro formato.

A continuación, se describen los tipos de datos fundamentales con los que se construyen las aplicaciones web:

- **String (cadena de texto)**

Cualquier información textual, una palabra, una frase o incluso un número que se procese como texto, como un código postal, se clasifica como *string*. Para que JavaScript lo interprete como texto y no como una instrucción, el contenido debe envolverse en comillas simples ' ', dobles " " o invertidas ` `.

Javascript

let nombreUsuario = "María";

let mensajeDeBienvenida = 'El usuario se ha conectado al chat.';

- **Number (número)**

A diferencia de otros lenguajes que separan los números enteros de los decimales, JavaScript agrupa todo bajo un mismo tipo. Sirven para operaciones matemáticas, conteos e intervalos de tiempo, y siempre se escriben sin comillas.

Javascript

let edad = 28;

let temperatura = 23.5; // Los decimales se marcan con punto

- **Boolean (booleano)**

Este tipo de dato es binario: solo puede tener uno de dos valores, true (verdadero) o false (falso). Funciona como el interruptor de luz de una habitación: encendido o apagado. Es fundamental para que el código tome decisiones lógicas.

Javascript

let estaConectado = true;

let tienePermisos = false;

- **Undefined y null (las cajas vacías)**

En ocasiones las variables no contienen información inmediata. JavaScript ofrece dos formas de representar esa ausencia:

- **Undefined*:*** la variable fue declarada pero no se le asignó ningún valor. Su contenido "no está definido".
- **Null*:*** se vacía la variable de forma deliberada, indicando explícitamente que "aquí no hay nada". Representa una ausencia de valor intencional.

- Cuando la información no se limita a un solo dato sino a un conjunto, por ejemplo, el nombre, la edad, el correo y las preferencias de un usuario simultáneamente, el desarrollo web recurre a tipos de datos estructurados: los `arrays` (listas ordenadas) y los objetos (estructuras detalladas), que se estudian en la sección 1.4.

1.3 Funciones y closures

Una función es una unidad de comportamiento que recibe entradas, realiza un proceso y puede devolver una salida. A nivel profesional, una función también cumple tres roles estructurales:

- Un mecanismo de abstracción (ocultar complejidad).
- Una herramienta de reutilización (evitar la repetición de lógica).
- Una forma de controlar el contexto (definir qué variables puede "ver" y qué estado puede modificar).

En JavaScript, las funciones son "ciudadanas de primera clase": pueden guardarse en variables, pasarse como argumentos y devolverse como resultados. Esto permite patrones potentes como los `callbacks` y las funciones de orden superior, pero también exige claridad para evitar confusión.

a) Contexto de ejecución

Cada vez que se ejecuta código, JavaScript crea un contexto de ejecución. Cada llamada a una función genera un contexto nuevo con su propio conjunto de variables locales, su "ambiente" y su flujo. Una buena analogía es imaginar cada función como un salón donde se realiza una actividad: dentro hay materiales (variables locales) que no se ven desde afuera, a menos que se permita explícitamente.

Este contexto incluye:

- Variables locales.
- Parámetros.
- Referencia al entorno externo (`scope chain`).
- Valor de **`this`** (que depende de cómo se invoca la función).

b) Alcance léxico y `closures`

JavaScript usa alcance léxico: una función puede acceder a las variables del lugar donde fue definida. Este comportamiento produce los `closures`.

Javascript

```
function crearContador () {
  let cuenta = 0;
  return function() {
    cuenta++;
    return cuenta;
  };
}
const contar = crearContador();
contar(); // 1
contar(); // 2
```

La función interna mantiene acceso a **cuenta** aunque **crearContador** ya terminó su ejecución: el *closure* preserva ese estado. Este mecanismo es clave para mantener estado privado sin exponerlo directamente, lo que garantiza el control de integridad del sistema.

c) El contexto de this

En JavaScript, **this** no es fijo: depende de cómo se invoca la función. Cuando el requisito es "código predecible", conviene evitar confusiones con **this** usando estrategias como las arrow functions (que no tienen **this** propio), el método **bind**, o el diseño con funciones puras.

El siguiente ejemplo aplica estos conceptos a un escenario de evaluación con requisitos técnicos concretos: reutilizar la lógica de validación, mantener un conteo de intentos sin exponerlo y generar retroalimentación clara.

```
Javascript
function crearEvaluador(respuestasCorrectas) {
let intentos = 0;
return function evaluar(respuestasAprendiz) {
intentos++;
// comparar respuestas...
return { intentos, resultado: "..." };
};
}
```

1.4 Objetos y estructuras de datos

En una aplicación real, como una plataforma educativa o un chat, la información rara vez viaja en forma de datos sueltos o simples cadenas de texto. Se requiere una forma de agrupar características complejas. Es aquí donde entran en juego los objetos.

En JavaScript, un objeto representa una entidad con propiedades (características) y comportamientos (acciones). A nivel de arquitectura de software, los objetos son la base para el modelado del dominio del problema: mediante ellos se representan

computacionalmente los usuarios, los cursos, las actividades, las evidencias y los mensajes.

Un objeto se puede representar como un archivero detallado de un aprendiz. Este archivero no solo tiene un nombre en la etiqueta, sino que en su interior contiene su edad, su correo, una lista con sus calificaciones e incluso las acciones que puede realizar en la plataforma.

- **El desafío de la mutabilidad: compartir vs. copiar**

Para utilizar estas estructuras con criterio técnico, se debe dominar un concepto crítico en JavaScript: la mutabilidad. Un objeto puede cambiar por dentro (mutar) sin que cambie la referencia principal que lo contiene. Esto puede ser útil para actualizar el estado de una aplicación en tiempo real, pero también resulta peligroso si genera cambios accidentales o inconsistencias.

Para ilustrar este comportamiento, en el siguiente ejemplo se describe el escenario:

```
Javascript
const grupoA = { nombre: "Grupo A", aprendices: [] }; const copia = grupoA;
// Se intenta agregar un aprendiz a la "copia"
copia.aprendices.push("Carlos");
```

Al hacer const **copia = grupoA**, no se crea un objeto nuevo, se comparte el acceso al mismo objeto original. Una analogía clara es un documento en Google Docs: si se comparte el enlace con un compañero, ambos están editando el mismo archivo. Si el compañero borra un párrafo, desaparece para todos. En JavaScript, si se ejecuta **copia.aprendices.push(...)**, el grupoA original también se verá alterado porque ambas variables apuntan a la misma dirección de memoria.

Para desarrollar software robusto, se necesita distinguir claramente entre:

- **Mutación intencional:** "actualizo el estado de la sesión activa porque el usuario hizo clic en guardar".
- **Mutación accidental:** "otro módulo del sistema cambió la lista de estudiantes sin avisar, alterando la visualización en pantalla".

- **Estrategias de clonación: copia superficial vs. copia profunda**

Cuando se necesita duplicar un archivero para trabajar en él sin alterar el original, existen dos niveles de profundidad:

- **Paso 1. Copia superficial (shallow copy)**

  Utilizando el operador de propagación o spread **({...objeto})**, se copian las propiedades del primer nivel. Sin embargo, si dentro del archivero hay un objeto o arreglo interno, ambas copias seguirán compartiendo la misma referencia a esa estructura interna.

- **Paso 2. Copia profunda (deep copy)**

  Es una clonación absoluta. Requiere una estrategia específica, como usar la función **structuredClone()**, transformar el objeto a texto y viceversa, o usar bibliotecas externas, para garantizar que toda la estructura, desde el exterior hasta el nivel más interno, sea completamente independiente.

- **Ejemplo en contexto educativo: tablero de participación**

  Para el diseño de un tablero de participación con los siguientes requisitos técnicos:

  - Mostrar las preguntas de los aprendices en tiempo real.
  - Evitar que una pregunta ya cerrada o calificada sea editada.
  - Mantener un historial inalterable para la evidencia del curso.

Si cada pregunta se modela como un objeto (const pregunta = { id, autor, texto, votos, estado }), la decisión técnica crucial es: ¿se muta la cantidad de votos directamente en el mismo objeto o se genera una versión nueva cada vez que alguien vota?

- Mutar es simple, requiere menos memoria y es rápido.
- Versionar (crear nuevos objetos inmutables) facilita la auditoría y reduce errores de concurrencia cuando cientos de aprendices votan simultáneamente.

En proyectos con requisitos estrictos de trazabilidad, como sistemas de educación, registros médicos o auditorías financieras, la mutabilidad descontrolada es un riesgo serio. Si las evidencias de aprendizaje pueden ser manipuladas accidentalmente en la memoria, el sistema pierde su validez. Por ello, ante cada objeto diseñado, un desarrollador con criterio profesional se detiene a preguntar: ¿Quién tiene el permiso real de cambiar esto? ¿Cómo se verifica que el cambio fue legítimo? ¿Qué evidencia queda de dicha modificación?

1.5 Estructuras de control y condicionales

Una vez comprendido cómo almacenar información en variables, agruparla en objetos y procesarla mediante funciones, surge una necesidad ineludible: dotar a la aplicación de "inteligencia" básica. Es aquí donde las estructuras de control y condicionales hacen su aparición. Si las funciones son los motores de una aplicación, los condicionales son el volante que decide hacia dónde debe girar ese motor según el entorno.

Un ejemplo concreto es el torniquete de entrada de una institución. El torniquete no actúa al azar; sigue una regla estricta: "si el carnet está activo, permitir el paso; de lo contrario, mantener la puerta bloqueada y emitir una alerta".

En el desarrollo de software educativo, los condicionales no son simplemente la sintaxis **if** o **else**; representan en código las políticas y normas del sistema. Constantemente se establecen reglas para:

- Habilitar o bloquear el acceso a un módulo.
- Permitir o impedir el envío de un formulario.
- Brindar retroalimentación positiva o negativa según el resultado de un cuestionario.
- Asignar rutas de aprendizaje personalizadas según el nivel del usuario.

- **Funcionamiento: claridad, legibilidad y mantenibilidad**

Escribir un condicional en JavaScript es técnicamente sencillo, pero el verdadero reto profesional radica en hacerlo comprensible. Un condicional puede volverse rápidamente ilegible si se encadenan múltiples reglas sin orden ni concierto, lo que en el gremio se conoce como **"código espagueti"**.

El desarrollo profesional siempre debe priorizar:

- La claridad de la condición: ¿es evidente lo que se está evaluando a simple vista?
- El manejo de casos límite: ¿qué sucede si el dato que llega no es un número o si el objeto está vacío?
- La relación explícita con el requisito: el código debe reflejar la regla de negocio.

Se debe desarrollar el hábito de escribir las condiciones como si fueran explicaciones en lenguaje natural. En lugar de pensar en abstractos cruces de variables, se programa dictando instrucciones claras: "si el aprendiz no está autenticado, negar acceso inmediatamente"; "si el puntaje es menor a 60, sugerir material de refuerzo".

- **Ejemplo en contexto educativo: habilitación de evaluación**

El siguiente flujo ilustra cómo convergen estos conceptos en la habilitación de una evaluación final. Los requisitos técnicos son:

  - Solo se permite iniciar la evaluación si se completó el contenido previo.
  - Se debe evitar que se salten pasos.
  - El sistema debe registrar los intentos de acceso.

a) **Paso 1**.

Verificar si el contenido fue completado (completo contenido).

b) **Paso 2.**

Si la condición es false: registrar el intento fallido y mostrar mensaje de orientación.

c) **Paso 3.**

Si la condición es **true**: habilitar la evaluación y notificar al aprendiz.

Javascript

```
const verificarAccesoEvaluacion = (estadoDelAprendiz) => {
if (!estadoDelAprendiz.completoContenido) {
registrarIntentoFallido(estadoDelAprendiz.id);
mostrarMensaje("Debe revisar y completar todo el material antes de habilitar la evaluación.");
} else {
```

```
habilitarEvaluacion(estadoDelAprendiz.id);
mostrarMensaje("¡La evaluación está habilitada!"); } };
}
    };
```

Lo fundamental no es memorizar la palabra clave **if**, sino la semántica detrás de ella. Al revisar este código, cualquier miembro del equipo debe poder afirmar: "este condicional implementa la regla funcional del sistema y protege la integridad del proceso de evaluación".

Los condicionales son, en esencia, decisiones que toma el software en nombre de sus creadores. Una regla condicional mal planteada o que emite mensajes de error confusos puede generar frustración y deserción. Por el contrario, una regla clara y diseñada con criterio ayuda a orientar, corrige el rumbo sin castigar y garantiza una experiencia de aprendizaje fluida y confiable.

1.6 Ciclos

Después de dotar a la aplicación de la capacidad de tomar decisiones mediante los condicionales, surge un nuevo desafío práctico: ¿qué sucede cuando una misma instrucción debe aplicarse no a un solo elemento, sino a cientos o miles de ellos? Es aquí donde entran en acción los ciclos o bucles (loops).

Un ciclo funciona como un proceso automatizado: se define una instrucción (tomar una hoja, sellarla y pasarla a la pila de terminados) y esta se repite hasta que la lista se agota.

En plataformas educativas, la información casi siempre se organiza en colecciones o listas:

- Una lista de aprendices matriculados.
- Un historial de mensajes en un chat grupal.
- Un repositorio de evidencias subidas a la plataforma.
- Un cronograma de sesiones virtuales.

El ciclo es la herramienta fundamental para recorrer y procesar esas listas.

- **Tipos de ciclos en JavaScript**
  - **For — el ciclo clásico con control total**

    Permite definir exactamente cuántas veces se ejecuta la instrucción. Es ideal cuando se conoce de antemano el número de iteraciones. Su estructura incluye inicio, condición y actualización en una sola línea.

    Javascript

    ```
    for (let i = 0; i < aprendices.length; i++) {
    console.log(aprendices[i]);
    }
    ```

  - **While — el ciclo condicional**

    Se ejecuta mientras una condición sea verdadera. Es útil cuando no se sabe cuántas iteraciones se necesitarán, por ejemplo, mientras haya mensajes pendientes en la cola del servidor.

    Javascript

    ```
    while (hayMensajesPendientes) {
    procesarSiguienteMensaje();
    }
    ```

- **ForEach — el ciclo moderno para listas**

  Método integrado en los arreglos que recorre cada elemento aplicando una función. Es más legible que el for clásico y se usa cuando no se necesita interrumpir el ciclo.

  Javascript

  ```
  aprendices.forEach(aprendiz => {
  enviarNotificacion(aprendiz.correo);
  });
  ```

- **Map — el ciclo que transforma**

  Similar a forEach, pero en lugar de solo recorrer, genera un nuevo arreglo con los resultados transformados. Es la herramienta correcta cuando se necesita convertir datos sin mutar el arreglo original.

  Javascript

  ```
  const correos = aprendices.map(aprendiz => aprendiz.correo);
  ```

- **Funcionamiento: eficiencia, límites y escalabilidad**

Node.js es increíblemente rápido, pero tiene un solo hilo principal de ejecución, hace una sola cosa a la vez a gran velocidad. Si se le asigna un ciclo pesado, se concentrará tanto en esa tarea que ignorará el resto de la aplicación.

Un ciclo que recorre una lista de 20 elementos es imperceptible. Pero un ciclo mal diseñado que recorre 20.000 registros en un momento crítico puede literalmente "congelar" la interfaz o saturar el servidor, provocando que los aprendices no puedan enviar mensajes ni cargar evidencias.

Por esta razón, frente a cualquier ciclo se debe incluir un análisis previo:

- ¿Cuántas iteraciones se van a ejecutar?

- ¿Qué operaciones hay dentro del ciclo? (¿se está consultando una base de datos en cada vuelta? Esto sería un error crítico).
- ¿Qué costo en memoria y tiempo tiene esta operación?
- ¿Existen alternativas más eficientes? (filtrar, paginar de 10 en 10 o indexar la información).

- **Ciclo tradicional vs. variable acumuladora**

  Ejemplo en contexto educativo: reporte de asistencia por grupo

  Para comparar un enfoque básico con uno optimizado, considérese el requerimiento de crear un reporte de asistencia. Los requisitos técnicos son:

  - Calcular el porcentaje de asistencia.
  - Generar un reporte visual.
  - Mantener tiempos de carga razonables.

  La solución inicial más lógica es un ciclo que recorra todas las marcaciones del mes:

  Javascript

```
let totalAsistencias = 0;
for (let i = 0; i < marcacionesDelMes.length; i++) {
  if (marcacionesDelMes[i].asistio === true) {
  totalAsistencias++;
  }
}
```

Sin embargo, si el requerimiento evoluciona a un reporte en vivo para un evento masivo con 1000 aprendices conectados simultáneamente, este ciclo se convierte en un cuello de botella.

En ese punto, se debe pensar estratégicamente: ¿Es necesario recalcular todo el historial cada vez que alguien entra? ¿No sería mejor usar una variable acumuladora que sume +1 cada vez que ocurre el evento de conexión, evitando el ciclo por completo?

En conclusión, un ciclo representa el poder de la repetición computacional. Sin embargo, en el desarrollo de aplicaciones web en tiempo real, la repetición excesiva y sin control se traduce en latencia (retrasos en el tiempo de respuesta del sistema). El criterio profesional no radica en saber escribir un ciclo, sino en determinar cuándo es la herramienta adecuada y cuándo la arquitectura exige una optimización para no comprometer la experiencia de los usuarios.

1.7. Declaración de funciones y código limpio

Luego de dominar cómo el código toma decisiones mediante condicionales y cómo procesa grandes volúmenes de información a través de los ciclos, se enfrenta a una revelación fundamental en el desarrollo profesional: hacer que el código funcione es apenas el primer paso; el verdadero reto es hacerlo comprensible.

En el entorno del desarrollo web existe una máxima inquebrantable: el código se escribe principalmente para que lo lean otros seres humanos, no solo para que lo ejecuten las máquinas. A la computadora le da exactamente igual si una función se llama x123 o si tiene mil líneas de largo; la máquina la traducirá y la ejecutará. Sin embargo, para el equipo de desarrollo que deba mantener esa plataforma en el futuro, o para el propio creador semanas después, leer un código desordenado es como intentar descifrar un texto sin signos de puntuación ni espacios.

La programación tiene una analogía directa con la redacción de un libro: el lenguaje de programación es la gramática, pero el clean code (código limpio) es el estilo, la ortografía y la coherencia de la historia.

- **El arte del nombramiento**

  Un nombre debe revelar la intención de quien lo escribió. No se necesitan comentarios extensos si los nombres son suficientemente descriptivos.

  - **Variables y constantes (los sustantivos)**

    Las variables guardan información y representan cosas o entidades. Sus nombres siempre deben ser sustantivos descriptivos.

    Ejemplo

    ❌ Incorrecto let n = "Carlos"; / const list = [ ];

    ✅ Correcto let nombreAprendiz = "Carlos"; / const listaCursos = [ ];

  - **Funciones (los verbos)**

    Las funciones realizan un trabajo y representan acciones. Sus nombres siempre deben comenzar con un verbo en infinitivo que indique exactamente qué hacen.

    Ejemplo

    ❌ Incorrecto const datos = () => {} / function usuario() {}

    ✅ Correcto const obtenerDatos = () => {} / function validarUsuario() {}

- **El principio de responsabilidad única**

  Una vez que las funciones tienen el nombre correcto, se debe evaluar su contenido. Aquí entra en juego uno de los principios más importantes de la ingeniería de software: el Single Responsibility Principle (principio de responsabilidad

única). Esta regla establece que una función debe hacer una sola cosa, y debe hacerla de manera excelente.

Un restaurante ilustra este principio con claridad: existe el rol del chef (cocinar) y el rol del mesero (atender las mesas). Si el chef tiene que soltar la espátula en medio de un pedido para ir a cobrarle a un cliente, el servicio colapsa. Cada rol debe tener una única responsabilidad.

En el software educativo ocurre lo mismo. Si se diseña una función llamada validarLogin para verificar la contraseña de un usuario, su único trabajo es decir "aprobado" o "rechazado". Si al revisar el código interno se descubre que esa misma función también envía correos electrónicos de bienvenida y guarda estadísticas en la base de datos, es el momento exacto para refactorizar (reestructurar el código sin cambiar su comportamiento externo).

2. Programación orientada a objetos (POO) en JavaScript

Una vez dominada la creación de objetos individuales y la escritura de funciones limpias para procesarlos, surge una pregunta logística inevitable en el desarrollo de aplicaciones reales: ¿qué sucede si la plataforma necesita registrar a 500 aprendices al mismo tiempo? Escribir manualmente 500 objetos idénticos con sus respectivas funciones sería inmanejable.

Para resolver este problema de escalabilidad, los lenguajes de programación utilizan la POO. Sin embargo, para comprender cómo funciona este enfoque en JavaScript, es necesario prepararse para desaprender y cambiar de paradigma, especialmente si se tiene experiencia previa en lenguajes más tradicionales como Java o C#.

JavaScript es un lenguaje único en su diseño: aunque incorpora una sintaxis de clases, estas no corresponden a clases tradicionales como en otros lenguajes, sino que funcionan como una abstracción sobre su modelo basado en prototipos.

- **El modelo de prototipos: el molde maestro**

En JavaScript, casi todo es un objeto. Cuando se crea un objeto nuevo, este no nace completamente aislado; nace con un enlace interno, una referencia oculta, que lo conecta a otro objeto anterior llamado su prototipo.

Este prototipo actúa como un molde maestro. Si se crea un objeto a partir de ese molde, el nuevo objeto hereda automáticamente todas las características y habilidades del original. A su vez, ese objeto molde tiene su propio prototipo superior, y así sucesivamente, formando una escalera conocida como la cadena de prototipos. Esta cadena asciende hasta que llega a un punto donde el prototipo es **null**, marcando el final del linaje.

- **Nivel 1**

  Objeto específico (el aprendiz).
- **Nivel 2**

  Prototipo directo (el mentor).
- **Nivel 3**

  Prototipo del prototipo (el coordinador).
- **Nivel 4**

  null (fin de la cadena).

- **El funcionamiento interno: pasando la pregunta hacia arriba**

  ¿Por qué JavaScript utiliza esta cadena invisible? La respuesta es eficiencia extrema en el consumo de recursos.

  Cuando se solicita a un objeto ejecutar la acción **saludar()**:
  - El motor de JavaScript primero revisa las propiedades propias del objeto específico. Si sabe cómo **saludar()**, lo hace.
  - Si no tiene esa habilidad directamente, no arroja un error inmediato. En su lugar, “pasa la pregunta hacia arriba” y revisa su prototipo.
  - Si el prototipo sabe **saludar()**, el objeto toma prestada esa habilidad y la ejecuta. Si tampoco sabe, se consulta el prototipo del prototipo, y así sucesivamente.

Esta delegación permite que cientos de objetos compartan los mismos métodos en la Memoria de Acceso Aleatorio (RAM) del servidor sin necesidad de duplicarlos, lo que reduce drásticamente el consumo de recursos.

2.1. Clases y prototipos

Si JavaScript usa prototipos, ¿por qué los desarrolladores modernos escriben la palabra **class** en su código?

Con la llegada de ECMAScript 2015 (ES6), los creadores del lenguaje decidieron introducir la palabra clave **class** para que los programadores provenientes de Java o C# se sintieran más cómodos y el código fuera más fácil de leer. Sin embargo, como señala Crockford (2008) en sus análisis sobre el lenguaje, esto es simplemente syntactic sugar (azúcar sintáctico).

El término azúcar sintáctico significa que se le ha puesto una envoltura familiar a la sintaxis, pero en el motor interno, JavaScript ignora la palabra **class** y sigue usando exactamente el mismo sistema de prototipos de siempre.

- **Anatomía de una clase en JavaScript**
  - **Declaración de la clase**

    Javascript

    // Se ve como una clase tradicional, pero es un prototipo oculto

    class Usuario {

    La palabra class es la envoltura visual. Internamente JavaScript la convierte en una función constructora con su cadena de prototipos.
  - **El constructor y las propiedades**

    Javascript

    constructor(nombre, correo) {

    this.nombre = nombre;

    this.correo = correo;

    }

El constructor es el molde maestro: define qué propiedades tendrá cada objeto creado a partir de esta clase; this hace referencia al objeto específico que se está creando en ese momento.

- **Los métodos (guardados en el prototipo)**

  Javascript

  ```
  iniciarSesion() {
      console.log(`${this.nombre} ha ingresado a la plataforma en tiempo real.`);
    }
  }
  ```

  Este método **no se copia** en cada objeto creado. JavaScript lo guarda automáticamente en el prototipo de **Usuario**, donde todos los objetos pueden consultarlo sin duplicarlo.

- **Prototipos vs. modelo ineficiente**

  Ejemplo en contexto educativo: eficiencia en memoria

Se crea la clase **Usuario** con el método **iniciarSesion()**. A partir de este molde, el sistema instancia 30 objetos de tipo aprendiz para un nuevo curso.

Modelo	**¿Qué ocurre en RAM?**
❌ Modelo ineficiente	30 copias exactas del método iniciarSesion() ocupando espacio
✅ Modelo de prototipos	30 aprendices ligeros compartiendo 1 solo método en el prototipo

Gracias al modelo de prototipos, los 30 aprendices no tienen el método repetido en su interior. Todos nacen ligeros y, cuando necesitan iniciar sesión, consultan hacia

arriba en su cadena de prototipos y hacen referencia al único método alojado en el prototipo general de **Usuario**.

Comprender esta diferencia no es solo un dato de teoría de la programación; es una competencia técnica indispensable. En el desarrollo de aplicaciones web en tiempo real con Node.js, donde un servidor puede gestionar miles de conexiones simultáneas por segundo, la RAM es el recurso más valioso. Al aprovechar la cadena de prototipos, se garantiza que la aplicación escale de manera robusta, compartiendo recursos de forma inteligente y evitando que el servidor colapse por redundancia de datos.

2.2 Nombramiento y organización del código

Tras comprender que las clases en JavaScript actúan como moldes maestros (basados en prototipos) para crear cientos de objetos de manera eficiente, surge una pregunta logística fundamental en el desarrollo de software: ¿cómo se organiza todo este código cuando el proyecto empieza a crecer?

- **Convenciones estrictas: el uniforme de las palabras**

  Para que un equipo de trabajo pueda leer el código de forma fluida, la industria del software ha establecido convenciones de escritura. Al ver una palabra, cualquier programador debe saber inmediatamente si se trata de un molde (una clase) o de una acción (un método).

  - **Clases (sustantivos en PascalCase)**

    Las clases son las plantillas o entidades principales del sistema. Sus nombres siempre deben ser sustantivos y escribirse en PascalCase: la primera letra de cada palabra va en mayúscula, sin espacios.

    ✅ Ejemplos correctos

GestorDeCalificaciones

AmbienteDeFormacion

UsuarioPremium

Al ver esa letra mayúscula inicial, el equipo sabe inmediatamente que está ante un molde maestro.

- **Métodos (verbos en camelCase)**

  Los métodos son las acciones que los objetos pueden realizar. Sus nombres siempre deben comenzar con un verbo en infinitivo y usar camelCase: la primera letra de la primera palabra es minúscula, pero las primeras letras de las palabras siguientes van en mayúscula, simulando las jorobas de un camello.

  ✅ Ejemplos correctos

  calcularPromedio()

  registrarAsistencia()

  enviarNotificacion()

- **Modularidad: una clase, un archivo**

Una vez que las clases y métodos tienen los nombres correctos, el siguiente paso es decidir dónde guardarlos. Es común que al inicio se escriba todo el proyecto en un solo y gigantesco archivo index.js, sin embargo, en un entorno de desarrollo real con Node.js, esto es una receta para el desastre.

La regla de oro de la organización en POO es la modularidad: una clase por archivo.

Para entender la importancia de esto, se puede imaginar un proyecto como un gran libro de actas físico. Si tres personas intentan escribir en la misma página

al mismo tiempo, chocarán entre sí y arruinarán el documento. En el desarrollo de software, esto ocurre cuando varios programadores trabajan en el mismo archivo usando repositorios de control de versiones como Git; el resultado es lo que se conoce como un merge conflict (conflicto de fusión).

Paso 1. Separar las clases por archivo

- Aprendiz.js → contiene única y exclusivamente la clase Aprendiz.
- Instructor.js → contiene solo la clase Instructor.
- Evaluacion.js → gestiona únicamente la lógica de las pruebas.

Paso 2. Trabajar en paralelo sin interferencias

Si se necesita actualizar la forma en que se calcula la asistencia de los aprendices y otra persona necesita agregar una función a los instructores, cada una trabaja en su propio archivo. No hay colisiones.

Paso 3. Integrar los aportes al sistema principal

Al finalizar, ambos aportes se integran al proyecto principal sin interferir en el trabajo del otro. El resultado es un sistema limpio, mantenible y listo para escalar.

2.3 Encapsulación, herencia y composición

Una vez que el proyecto cuenta con clases bien nombradas y archivos organizados de manera modular, el desarrollo de aplicaciones web sube de nivel. Ya no se trata solo de que el código funcione o se lea bien, sino de protegerlo y hacerlo verdaderamente escalable. Aquí es donde se establece la diferencia entre un codificador novato y un verdadero arquitecto de software.

Para lograr esta madurez técnica, se deben dominar tres pilares fundamentales de la POO: la encapsulación, la herencia y la composición.

- **Los tres pilares de la POO**
  - **La encapsulación: el principio de la caja negra**

    En un entorno colaborativo o en una aplicación en tiempo real, múltiples partes del sistema interactúan constantemente. La encapsulación es el mecanismo de seguridad diseñado para proteger el estado interno de un objeto, evitando que otras partes del programa lo modifiquen accidentalmente o sin autorización.

    Un ejemplo es el sistema de matrículas: el objeto **Matricula** guarda en su interior un dato muy sensible: el resultado de la evaluación del aprendiz. Si el sistema no está encapsulado, cualquier fragmento de código podría alterar ese resultado de forma directa y arbitraria con una simple línea: **aprendiz.resultado = "A"**. Esto es un riesgo inaceptable, pues comprometería la validez e integridad de las evidencias de aprendizaje.

    En las versiones modernas de JavaScript, la encapsulación se logra utilizando el prefijo # (numeral) antes del nombre de la propiedad o método para declararlo como privado. Esto convierte al objeto en una "caja negra"; su interior es inaccesible desde el exterior.

    Las aduanas: `getters` y `setters`

  - **`Getter`** (para obtener): si la propiedad es privada, la única forma de consultarla desde el exterior es a través de un método público controlado llamado `getter`. Este actúa como una ventanilla de consulta: entrega el resultado, pero no permite modificarlo directamente.
  - **`Setter`** (para modificar): el *setter* es la "aduana" del objeto. Antes de permitir que el resultado cambie, verifica quién está intentando hacer el cambio, si el

valor es válido (solo "A" o "D") y si la fecha límite no ha pasado. Solo si todo es correcto, la aduana permite el cambio.

- **Herencia vs. composición: el dilema del gorila y la banana**

  Árbol genealógico vs. bloques de Lego

  Una vez que los objetos están seguros, el siguiente paso es reutilizar código. Históricamente el enfoque principal ha sido la herencia.

  La herencia (implementada en JavaScript moderno con la palabra clave extends) permite crear relaciones del tipo "es un/a". Por ejemplo, tiene sentido lógico decir que "un instructor es un usuario". Por lo tanto, la clase Instructor hereda todas las propiedades y métodos de la clase padre Usuario, añadiendo únicamente sus características específicas.

  Sin embargo, a medida que las aplicaciones crecen, las jerarquías de herencia profundas crean un código rígido y frágil. En la ingeniería de software existe un famoso dilema conocido como el problema del gorila y la banana, formulado por Armstrong (2019): "querías una banana, pero te llevaste un gorila sosteniendo la banana y toda la selva con él".

  Esto ocurre cuando se crea una nueva clase y se le hace heredar de una clase gigante, por ejemplo, InstructorTecnico hereda de Profesor, que hereda de Empleado, que hereda de Usuario, solo porque se necesitaba reutilizar un pequeño método como saludar(). Como resultado, el nuevo objeto carga en memoria con funciones y configuraciones que jamás va a utilizar, afectando el rendimiento de la aplicación en tiempo real.

- **La alternativa del arquitecto: la composición**

  Mientras que la herencia se basa en lo que un objeto **es**, la composición se basa en lo que un objeto **tiene** o **sabe hacer**. En lugar de construir un árbol

genealógico rígido, la composición se asemeja a construir con bloques de Lego.

Si se necesita crear un **InstructorTecnico**, en lugar de obligarlo a heredar de una larga cadena de clases, el sistema crea pequeños módulos independientes de comportamiento: un **moduloAutenticable** (que sabe iniciar sesión) y un **moduloEvaluador** (que sabe calificar). Luego se compone el objeto final inyectándole únicamente las habilidades que necesita.

Este enfoque garantiza un software altamente modular, fácil de modificar, escalable y eficiente en el consumo de recursos del servidor, una característica vital cuando se trabaja con Node.js.

3. Node.js para tiempo real

Hasta este punto se han dominado las bases estructurales de JavaScript: cómo guardar datos, procesarlos con funciones limpias, estructurarlos en objetos seguros y organizarlos con POO. Ahora es el momento de llevar todo ese conocimiento a un nuevo escenario: el servidor.

Es aquí donde entra en escena Node.js. Para comprender sus características y beneficios, se debe entender primero qué es exactamente y para qué fue diseñado.

Node.js no es un lenguaje de programación nuevo; es un entorno que permite ejecutar JavaScript fuera del navegador web. Sin embargo, su arquitectura tiene un propósito muy específico. Node.js no fue diseñado para realizar cálculos matemáticos pesados. Si se le pide renderizar un video en 3D o procesar complejos algoritmos de inteligencia artificial, su rendimiento será deficiente. Su verdadero poder, y la razón por la que domina la web moderna, es su capacidad para resolver un reto histórico de internet conocido como el problema **C10K**: la capacidad de manejar 10.000 o más conexiones de red simultáneas sin que la máquina colapse.

3.1 Características y beneficios

- **I/O no bloqueante (non-blocking I/O): el secreto de la velocidad**

  Para entender cómo Node.js logra esta hazaña, es necesario compararlo con la forma en que operaban los servidores web tradicionales (como los construidos con PHP o las versiones clásicas de Java).

  En el modelo tradicional, cada vez que un usuario se conecta a la página web, el servidor contrata a un nuevo "trabajador" (conocido técnicamente como un hilo o `thread`) para atender exclusivamente a esa persona. Si el usuario pide una lista de calificaciones, el trabajador va a la base de datos (una operación de

entrada/salida o I/O) y se queda allí, de brazos cruzados y bloqueado, esperando pacientemente a que la base de datos responda. Si 10.000 estudiantes entran al mismo tiempo, el servidor debe contratar 10.000 trabajadores, lo que agota rápidamente la memoria RAM y hace que el sistema se caiga.

Node.js cambia las reglas del juego utilizando un modelo de I/O no bloqueante. En lugar de tener miles de trabajadores perezosos, Node.js tiene un solo "hilo principal" (un único trabajador) que nunca, en ninguna circunstancia, se detiene a esperar.

Las operaciones de leer un archivo o consultar una base de datos son computacionalmente "lentas". En Node.js, cuando llega una de estas peticiones, el hilo principal la delega inmediatamente al sistema operativo y continúa atendiendo a la velocidad de la luz a los miles de usuarios restantes.

- **La analogía del restaurante de alta cocina**

Este chef (que representa el hilo principal o `event loop`) es el único encargado de recibir los pedidos. Él no se pone a picar cebollas, ni baja al sótano a buscar ingredientes, ni lava los platos; esas son tareas lentas.

a) El chef recibe una orden de un mesero (una petición HTTP de un usuario).

b) Inmediatamente, delega esa tarea pesada a sus ayudantes de cocina (un conjunto de herramientas internas en Node.js conocido como `libuv` y el propio sistema operativo).

c) Sin perder un milisegundo, el chef se da la vuelta y recibe la orden del siguiente mesero.

d) Mientras tanto, los ayudantes están en segundo plano picando la cebolla (buscando en la base de datos).

e) Cuando un ayudante termina, no interrumpe al chef a gritos. Le deja una nota o le da un ligero toque en el hombro indicando que el ingrediente está listo.

f) En su primer momento libre, el chef toma ese ingrediente, ensambla el plato final y lo entrega al cliente.

- **Beneficios indiscutibles para el tiempo real**

En las aplicaciones web en tiempo real, la respuesta a por qué Node.js es el rey del "tiempo real" se vuelve evidente.

Las aplicaciones web en tiempo real, como los chats de soporte, las pizarras colaborativas donde varios aprendices dibujan al mismo tiempo, o los tableros de notificaciones en vivo, comparten una característica: generan miles de conexiones simultáneas que envían mensajes muy pequeños de forma constante.

En estas aplicaciones, no hay cálculos matemáticos pesados; hay una avalancha de pequeñas operaciones de entrada/salida (I/O). Node.js, gracias a su chef ejecutivo que no se bloquea y a su dominio absoluto de la asincronía, procesa estos miles de microinteracciones con una huella de memoria mínima y una latencia casi nula.

3.2 Bibliotecas y ecosistema npm

Tras entender que Node.js actúa como un chef ejecutivo sumamente veloz que no se bloquea ante múltiples peticiones, surge una duda práctica: ¿tiene este chef que inventar sus propios utensilios y recetas desde cero cada vez? La respuesta es un rotundo no. Aquí es donde entra en juego el ecosistema que catapultó a Node.js al éxito mundial.

El gestor de paquetes de Node (Node Package Manager, npm) es el registro de software de código abierto más grande del mundo. Su filosofía se fundamenta en

un principio clásico de los sistemas Unix: "escribe programas que hagan una sola cosa, pero que la hagan increíblemente bien".

En plataformas del pasado, era común crear entornos monolíticos: aplicaciones gigantescas donde un solo equipo debía programar absolutamente todo, desde la conexión a la base de datos hasta el sistema de contraseñas. En Node.js, el paradigma cambia drásticamente; los desarrolladores ensamblan sus aplicaciones uniendo pequeñas y eficientes piezas de código abierto construidas por la comunidad global.

- **La analogía de la ferretería global y los bloques de Lego**

Npm funciona como una ferretería de tamaño global y acceso gratuito: si en la plataforma educativa se necesita un módulo para generar archivos PDF con los resultados de los aprendices, el desarrollador no invierte meses estudiando las especificaciones técnicas del formato PDF. Simplemente "va a la ferretería" (npm), busca el bloque diseñado específicamente para crear PDFs, lo descarga con un solo comando en su terminal y lo ensambla en su proyecto.

Para las aplicaciones en tiempo real, este ecosistema es invaluable. Implementar conexiones persistentes a través de `WebSockets` puros desde cero implica manejar protocolos de red complejos a muy bajo nivel. Gracias a npm, el desarrollador puede apoyarse en bibliotecas especializadas como socket.io, que resuelven toda esa complejidad, permitiendo levantar un servidor de chat bidireccional en cuestión de minutos.

- **Evaluación crítica: el criterio de selección profesional**

Sin embargo, tener acceso gratuito a millones de herramientas conlleva una responsabilidad enorme. Un desarrollador *junior* podría sentirse tentado a instalar paquetes de forma indiscriminada para cualquier tarea menor. Un

desarrollador *senior* sabe que cada paquete añadido es código de terceros ejecutándose en su servidor.

Por ello, se debe aprender a auditar y seleccionar estas herramientas con rigor. No se instala un paquete solo porque aparece en el primer resultado de búsqueda.

- **Criterios de selección profesional**
  - **Popularidad (descargas semanales)**

    Si una biblioteca tiene tres millones de descargas a la semana, está siendo utilizada y probada constantemente por miles de empresas. Si tiene diez descargas, es un riesgo no probado. ¿Cuántos equipos confían en esta herramienta?
  - **Estabilidad (issues en GitHub)**

    Al revisar el repositorio público del código, ¿cuántos problemas han reportado otros usuarios? Y más importante: ¿los creadores los están resolviendo activamente? Un repositorio con cientos de issues sin respuesta es una señal de alerta.
  - **Mantenimiento (fecha del último commit)**

    La tecnología web evoluciona a diario. Si la última actualización del paquete fue hace dos años, se considera una biblioteca abandonada (abandonware) y representa un riesgo crítico de seguridad cibernética.
  - **Licenciamiento legal**

    Es un error creer que todo el código abierto es de uso libre sin restricciones. Se debe distinguir entre licencias permisivas (como la licencia MIT) y licencias estrictas (como GPL, que obligan a abrir el código fuente derivado). Ignorar esto expone a futuros empleadores a demandas legales.

Depender de una biblioteca abandonada o de dudosa procedencia no solo genera incompatibilidades futuras, sino que representa un riesgo crítico de seguridad. Las vulnerabilidades no parcheadas en dependencias de terceros son una de las principales puertas de entrada para ataques informáticos modernos conocidos como ataques a la cadena de suministro.

- **Ejemplo en contexto educativo: fechas en un chat en vivo**
  - **El enfoque manual (propenso a errores)**

    Imagine que el sistema requiere mostrar en el chat colaborativo cuánto tiempo hace que un aprendiz envió un mensaje (por ejemplo, "Hace 5 minutos"). El desarrollador podría intentar escribir una función manual que calcule años bisiestos, meses de 31 días y zonas horarias. El resultado: código extenso, frágil y propenso a errores difíciles de depurar.
  - **El enfoque profesional (bibliotecas especializadas)**

    La alternativa profesional es buscar en npm una biblioteca especializada en manejo de fechas como **date-fns** o **moment**. Se instala con un solo comando, hace "una sola cosa y la hace bien", y permite concentrarse en lo verdaderamente importante: la lógica de la aplicación.

3.3 Conceptos de TCP y sockets

Tras levantar un servidor robusto con Node.js y las herramientas del ecosistema npm, se llega frente al último gran obstáculo conceptual para construir la aplicación. Hasta este punto, la web se ha comportado bajo unas reglas muy estrictas: el modelo de comunicación HTTP.

Para que una aplicación educativa sea verdaderamente de tiempo real, como un chat de asesoría sincrónica o un tablero colaborativo donde varios aprendices dibujan

al mismo tiempo, el modelo HTTP tradicional de petición y respuesta, lamentablemente, fracasa.

- **El problema de HTTP: una conversación unidireccional y sin memoria**

El protocolo HTTP es, por diseño, un protocolo `stateless` (sin estado) y estrictamente unidireccional. Esto significa que el servidor web es un ente pasivo; jamás iniciará una conversación. El cliente (el navegador) siempre debe dar el primer paso y hacer una petición. El servidor procesa, entrega la respuesta y de inmediato "cuelga el teléfono" y olvida quién era ese cliente.

La comunicación HTTP se puede comparar con el envío de una carta postal: se envía una carta preguntando "¿El instructor ya calificó mi evidencia?". El servidor busca, responde "No", y la conexión se cierra. Si un minuto después se quiere saber si el resultado ya está disponible, la única forma es enviar otra carta completa o recargar la página entera.

En el pasado, para simular el tiempo real, los programadores utilizaban una técnica llamada `short polling` (sondeo corto): programar el navegador para que enviara una petición HTTP automáticamente cada tres segundos preguntando: "¿Hay mensajes nuevos?". Esto es el equivalente informático a un niño en un viaje preguntando cada cinco minutos: "¿Ya llegamos?". La mayoría de las veces el servidor responde que no, desperdiciando ancho de banda y saturando la red innecesariamente.

- **El funcionamiento: el `handshake` y el `upgrade`**

¿Cómo se pasa de una página web normal (HTTP) a esta llamada telefónica constante? Todo ocurre mediante un proceso de negociación inicial muy elegante.

a) **La petición inicial**

El navegador hace una petición HTTP normal, pero lleva una etiqueta especial solicitando un `upgrade` (una mejora de conexión). Es como decirle al

recepcionista: "Vengo por la vía tradicional, pero ¿podemos pasarnos a una línea telefónica directa?".

b) **La aceptación**

Si el servidor de Node.js está configurado para `WebSockets`, acepta la solicitud y confirma el `upgrade`.

c) **La transformación**

En ese milisegundo, el canal HTTP se transforma en un túnel TCP abierto permanentemente. A partir de ese momento, ya no es necesario recargar la página ni hacer consultas constantes. Ahora el servidor tiene el poder de empujar (`push`) las notificaciones directamente al cliente en tiempo real.

3.4 Ciclo de vida y `event loop`

Habiendo comprendido que los `WebSockets` mantienen cientos o miles de túneles de comunicación abiertos de forma permanente, surge una pregunta obligada: ¿cómo hace el servidor para escuchar a todos esos usuarios al mismo tiempo sin colapsar?

La respuesta reside en el concepto técnico más importante de cualquier desarrollador `backend` en el ecosistema de JavaScript: el `event loop` (ciclo de eventos).

Como se mencionó anteriormente, Node.js es single-threaded (tiene un solo hilo principal). Sin embargo, logra ser altamente concurrente y manejar miles de tareas gracias a este ciclo infinito y a una potente biblioteca interna escrita en C++ llamada libuv.

El `event loop` es un ciclo infinito que procesa operaciones en fases estrictamente ordenadas. Cada fase tiene una responsabilidad específica y, una vez

completada la última, el ciclo vuelve a empezar desde la primera, de forma continua y a gran velocidad.

- **Las fases del ciclo**
  - **Nivel 1**

    Timers (temporizadores)

    El ciclo comienza mirando el reloj. En esta fase se ejecutan las funciones programadas para un momento específico usando setTimeout() o setInterval(). Es el equivalente a que el instructor diga: "Son las 10:00 AM, es hora de recoger el examen de los que tenían tiempo extra".
  - **Nivel 2**

    Callbacks pendientes

    Aquí se resuelven las tareas diferidas por el sistema operativo, como un error de red al intentar conectarse a la base de datos. Es como revisar si quedó algún asunto pendiente antes de avanzar.
  - **Nivel 3**

    Idle, prepare (inactividad y preparación)

    Fase de uso interno exclusivo del motor de Node.js. En la analogía, es el microsegundo en el que el instructor organiza sus hojas en el escritorio antes de abrir la puerta.
  - **Nivel 4**

    Poll (sondeo - el trabajo real)

    Esta es la fase más importante. Aquí Node.js espera a que lleguen nuevas peticiones: conexiones de WebSockets, mensajes del chat, lectura de archivos. Si hay tareas en la fila, el sistema las ejecuta una por una hasta que la sala de espera se vacía.

- **Nivel 5**

  Check (revisión)

  Una vez atendidas todas las tareas del sondeo, se ejecutan las funciones marcadas como urgentes mediante setImmediate(). Es como cuando el instructor dice: "Justo antes de terminar este ciclo, necesito que firmen esta asistencia de inmediato".

- **Nivel 6**

  Close callbacks (eventos de cierre)

  Finalmente, el ciclo limpia la casa. Ejecuta los eventos de cierre brusco, como cuando un usuario cierra su navegador de golpe (socket.on('close', ...)). El ciclo despide a los que se van y vuelve a empezar en la fase 1.

- **Rigor técnico: el peligro de bloquear la puerta**

Comprender esta coreografía es lo que separa a un desarrollador que solo copia código de uno que diseña sistemas robustos.

El mayor riesgo en Node.js es olvidar que solo hay un hilo. Si se comete el error de escribir un ciclo **for** masivo (por ejemplo, contar de uno a mil millones) o una operación matemática pesada como encriptar contraseñas directamente en el hilo principal, ocurrirá un desastre: se bloqueará el event loop.

Si el ciclo queda paralizado procesando una operación pesada, significa que la puerta del servidor queda bloqueada: no se podrá aceptar ni una sola petición nueva, los mensajes no se enviarán y la plataforma entera parecerá estar caída para miles de usuarios simultáneos.

En un entorno profesional, este problema se mitiga con rigor técnico: las tareas matemáticas o criptográficas pesadas jamás se dejan en el hilo principal. Se deben

delegar a "ayudantes externos" utilizando worker threads, permitiendo que el event loop siga girando libremente y atendiendo el tráfico en tiempo real.

Para complementar la comprensión del event loop y el modelo de I/O no bloqueante, se sugiere explorar el siguiente recurso de audio: El event loop y el modelo de I/O no bloqueante en Node.js.

Transcripción del pódcast. El event loop y el modelo de I/O no bloqueante en Node.js.
– **Hombre:** imaginen que entran a su cafetería favorita, esa donde el olor a café recién molido los hace sentir de inmediato como en casa. – **Mujer:** pero hoy el lugar está a reventar. Mesas ocupadas, una fila enorme que llega hasta la puerta y, sin embargo, hay algo que no encaja. – **Hombre:** solo hay una persona atendiendo. Una sola persona encargada de cada pedido, de cada proceso. El lugar debería ser un caos, sin embargo, todo funciona y transcurre con normalidad. – **Mujer:** y es inevitable preguntarse, ¿cómo lo hace? ¿Cómo evita el caos total y logra que todo funcione a la perfección? La respuesta es más simple de lo que parece: el cajero no realiza una a una las distintas tareas que se requieren, sino que registra y continúa con la siguiente en un proceso ágil y eficiente. – **Hombre:** él no se queda mirando la cafetera hasta que salga el tinto. Él toma el pedido, el tiquete, y antes de que el cliente guarde la billetera, ya está atendiendo al siguiente. – **Mujer:** ese nivel de eficiencia es el event loop de Node.js y lo mejor es que opera de la misma manera: en un solo hilo de procesamiento. Como en este caso, una

sola persona coordinando todo en la cafetería. Su secreto es que nunca se queda quieto esperando.

- **Hombre:** exacto. El cajero encarga y continúa, dejando que sea el sistema operativo quien haga el trabajo pesado en segundo plano. Técnicamente es lo que llamamos entrada y salida no bloqueante.
- **Mujer:** node no se queda congelado esperando una respuesta del disco o la red, registra la tarea, libera el hilo único y salta de inmediato a la siguiente petición.
- **Hombre:** pero aquí está el secreto técnico. Ese tiquete de pedido es el que nos garantiza que la orden no se pierda. En programación lo llamamos `callback`.
- **Mujer:** de ahí parte la base de la entrada y salida no bloqueante. Mientras el sistema operativo gestiona tareas pesadas en segundo plano como leer un archivo o consultar una base de datos, el hilo único de Node queda libre para recibir miles de peticiones nuevas.
- **Hombre:** pero hay que ser precavidos, la eficiencia tiene un límite. Es como si le pidiéramos al cajero que deje de atender para ponerse a contar los granos de café uno por uno, ahí sí se bloquea la fila y el sistema colapsa.
- **Mujer:** aquí en Node ocurre lo mismo. Sí se satura el hilo principal con procesos pesados, el `event loop` se detiene. Para esas tareas que requieren más trabajo lo correcto es usar `worker threads` y mantener el lugar siempre despejado.
- **Hombre:** comprender el comportamiento de este hilo único es el paso definitivo para construir sistemas capaces de crecer y sostenerse sin importar el volumen de peticiones que reciban.

- **Mujer:** si están listos para pasar de tirar código a diseñar sistemas de alto nivel, los invitamos a revisar el material del componente formativo para profundizar en los detalles únicos.
- **Hombre:** gracias por acompañarnos a tomarnos este cafecito y aprender más sobre los múltiples beneficios de aplicar Node. Nos escuchamos en el próximo programa.

3.5 Instalación y verificación del entorno

En un contexto profesional, rara vez se instala Node.js directamente desde el instalador global del sistema operativo. La práctica estándar en la industria es utilizar gestores de versiones como el gestor de versiones de Node (Node Version Manager, NVM).

- **Instalación profesional del entorno**

  a) Instalar NVM

  NVM permite gestionar múltiples versiones de Node.js en la misma máquina. Un desarrollador podría estar trabajando en el proyecto A (que requiere Node v14) y en el proyecto B (que exige Node v20). NVM permite cambiar la versión activa del motor en la terminal en milisegundos, garantizando la compatibilidad del entorno sin reinstalar nada.

  bash

  # Comando de instalación en sistemas Unix/Linux

  curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash

  b) Instalar la versión LTS de Node.js

  Se recomienda utilizar siempre versiones de soporte a largo plazo (Long Term Support, LTS) para proyectos empresariales, dejando las versiones current

(actuales) solo para experimentación de nuevas características. Las versiones LTS garantizan estabilidad, parches de seguridad y soporte extendido.

```
bash
# Instalar la versión LTS más reciente
nvm install --lts
# Activar la versión LTS instalada
nvm use --lts
```

c) **Verificar la instalación**

Una vez completada la instalación, se debe verificar que el entorno quedó correctamente configurado. Estos comandos confirman que tanto Node.js como npm están disponibles y operativos:

```
bash
# Verificar versión de Node.js instalada
node --version
# Verificar versión de NPM instalada
npm --version
```

Si ambos comandos retornan un número de versión, el entorno está listo para el desarrollo.

3.6 Gestión de dependencias, seguridad y mantenimiento

Una vez instalado el entorno, el siguiente paso es entender cómo Node.js organiza y protege las dependencias del proyecto. El archivo package.json es el corazón organizativo de cualquier proyecto: define los `scripts` de ejecución, los metadatos del sistema y las dependencias (bibliotecas de terceros) necesarias para su funcionamiento.

- **Los tres pilares de la gestión del proyecto**
  - **Package.json: el corazón del proyecto**

    El archivo package.json centraliza toda la información del proyecto en un solo lugar. Define:

    El nombre, versión y descripción del sistema.

    Los `scripts` de ejecución (por ejemplo, cómo iniciar el servidor o ejecutar las pruebas).

    Las dependencias necesarias para producción y las dependencias de desarrollo.

    json

```
{
  "name": "plataforma-educativa",
  "version": "1.0.0",
  "scripts": {
     "start": "node index.js",
     "dev": "nodemon index.js"
},
"dependencies": {
   "express": "^4.18.2",
   "socket.io": "^4.6.1"
 }
}
```

  - **Seguridad: npm audit**

    En la cultura del desarrollo web, la seguridad no es opcional. El comando **npm audit** escanea automáticamente el árbol de dependencias (**node_modules**)

cruzándolo con bases de datos globales de vulnerabilidades y exposiciones comunes (Common Vulnerabilities and Exposures, CVE), alertando sobre riesgos como inyecciones de lenguaje de consulta estructurado (Structured Query Language, SQL) o vulnerabilidades de denegación de servicio (DoS) en las bibliotecas utilizadas.

bash

```
# Escanear vulnerabilidades en las dependencias
npm audit
# Corregir automáticamente las vulnerabilidades detectadas
npm audit fix
```

Un proyecto profesional ejecuta npm audit de forma periódica y antes de cada despliegue a producción. Las vulnerabilidades no parcheadas en dependencias de terceros son una de las principales puertas de entrada para ataques informáticos modernos.

- **Sostenibilidad del software: package-lock.json**

  El archivo package-lock.json garantiza que todos los desarrolladores del equipo, así como el servidor de producción, instalen las versiones matemáticamente exactas de cada dependencia, evitando el problema conocido como "en mi máquina sí funciona".

Tabla 1. Comandos de instalación según entorno de desarrollo

Comando	Cómo usarlo
npm install	Desarrollo local - permite actualizaciones menores.
npm ci	Producción - instalación exacta basada en package-lock.json.

Nota

El comando **npm ci** (clean install) lee exclusivamente el archivo **package-lock.json**, garantizando instalaciones reproducibles en cualquier entorno.

4. Caracterizar y seleccionar herramientas

En el entorno productivo real no existe una "herramienta perfecta" universal. Existe la herramienta adecuada para el contexto específico. Se debe transitar de un consumidor pasivo de tutoriales a un arquitecto de software capaz de auditar, caracterizar y seleccionar tecnologías con criterio profesional.

4.1. Criterios de caracterización

Cuando se enfrenta el ecosistema de npm, que cuenta con millones de paquetes, elegir una biblioteca al azar para solucionar un problema es un riesgo inaceptable. Para caracterizar y seleccionar una herramienta en JavaScript de forma profesional, se debe aplicar una matriz de evaluación estricta.

- **Criterios de caracterización profesional**
  - **Mantenibilidad y salud del repositorio**

    ¿Cuándo fue la última actualización (commit)?

    Si un libro de texto no se ha revisado en cinco años, su información es obsoleta. En el software, una biblioteca que no ha sido actualizada en dos años se considera código muerto (abandonware) y representa un riesgo crítico de seguridad cibernética.

    Pregunta clave: ¿cuándo fue el último commit y con qué frecuencia se actualiza?
  - **Comunidad y documentación**

    Si el manual de un equipo de laboratorio está en un idioma desconocido o mal redactado, el tiempo que tardará el equipo en usarlo será altísimo. De igual manera, la curva de aprendizaje de un equipo de programación depende

directamente de la calidad de la documentación oficial. Un manual pobre dispara el costo en horas de desarrollo.

Pregunta clave: ¿la documentación es clara, completa y está actualizada?

- **Impacto en el rendimiento (peso de la herramienta)**

  En el desarrollo web, la velocidad lo es todo y cada kilobyte cuenta. Importar una biblioteca masiva de 2 MB solo para utilizar una pequeña función que formatea fechas es una negligencia técnica que ralentiza la aplicación. En Node.js, donde el rendimiento es crítico, el peso de cada dependencia debe justificarse.

  Pregunta clave: ¿el tamaño de la biblioteca se justifica con las funcionalidades que aporta?

- **Licenciamiento legal**

  Es un error común creer que todo el código abierto es de uso libre sin restricciones. Se debe distinguir entre licencias permisivas, como la licencia del Instituto Tecnológico de Massachusetts (Massachusetts Institute of Technology, MIT), y licencias estrictas, como la Licencia Pública General (General Public License, GPL), que obliga a abrir el código fuente derivado. Ignorar esto expone a futuros empleadores a demandas legales.

  Pregunta clave: ¿la licencia de esta biblioteca es compatible con el uso comercial del proyecto?

4.2 Matriz de decisión para tiempo real

Para facilitar la selección de herramientas de comunicación en tiempo real, se presenta la siguiente matriz de decisión técnica:

Tabla 2. Matriz de decisión para herramientas de comunicación en tiempo real

Escenario del proyecto	Tecnología recomendada	Justificación técnica	Limitaciones
Chat sencillo / notificaciones básicas	socket.io	Proporciona fallbacks (alternativas automáticas) a long polling si el navegador del cliente es muy antiguo. Maneja reconexiones automáticas y salas de chat (rooms) de forma nativa.	Agrega peso adicional al cliente y al servidor. No es el protocolo crudo.
Streaming de datos financieros (alta frecuencia)	ws (WebSockets nativos)	Es la implementación más ligera y rápida de WebSockets para Node.js. Mínima latencia, ideal para miles de mensajes por segundo.	El desarrollador debe programar manualmente la lógica de reconexión y emisión estructurada.
Dashboard unidireccional (solo el servidor envía datos)	Eventos enviados por el servidor (Server-Sent Events, SSE)	Utiliza el protocolo HTTP tradicional. Es extremadamente eficiente cuando el cliente solo necesita escuchar datos en tiempo real.	Es unidireccional (servidor → cliente). No sirve para comunicación bidireccional.

Nota

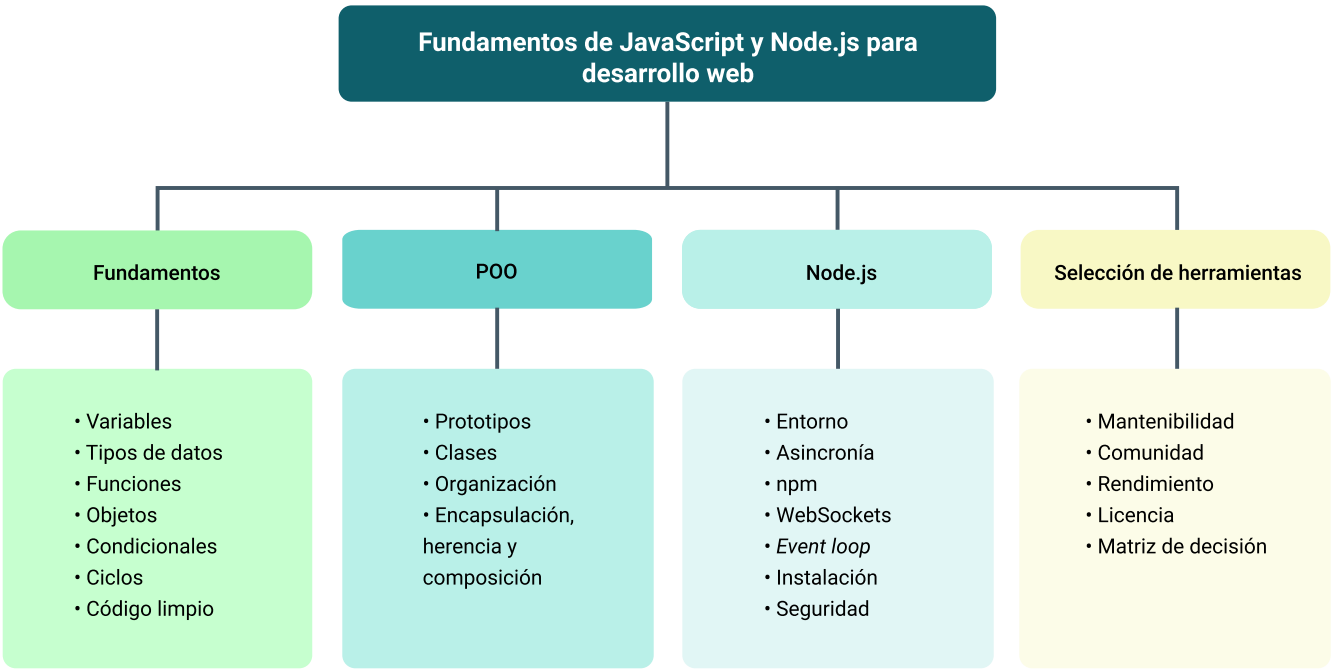
La selección de la tecnología debe basarse siempre en los requisitos específicos del proyecto, el perfil del usuario final y las restricciones de infraestructura disponible.

En el entorno productivo real, no existe una tecnología superior a las demás en términos absolutos. La diferencia entre un desarrollador competente y uno excepcional radica en su capacidad para leer los requisitos del sistema, contrastarlos con las características de cada herramienta y tomar una decisión argumentada, documentada y revisable.

Síntesis

La construcción de aplicaciones web en tiempo real requiere una comprensión integral que articule los fundamentos lógicos de JavaScript, el modelado estructurado de la información y las dinámicas de un código limpio y modular. Reconocer el software como una construcción diseñada para equipos humanos, y no únicamente para máquinas, permite analizar su arquitectura con mayor objetividad, fortalecer la toma de decisiones técnicas y promover soluciones escalables que favorezcan la eficiencia del sistema y el trabajo colaborativo.

Asimismo, el dominio del entorno de servidor con Node.js, el manejo de la concurrencia asíncrona y la implementación de conexiones persistentes confirman que el rendimiento es el pilar fundamental de la interactividad. Asumir un rol analítico y crítico frente a la evaluación de herramientas tecnológicas de terceros permite estructurar proyectos con rigor profesional, garantizando plataformas seguras, estables y plenamente alineadas con las exigencias de los usuarios y del ecosistema digital moderno.


Fundamentos de JavaScript y Node.js para desarrollo web
Fundamentos
• Variables
• Tipos de datos
• Funciones
• Objetos
• Condicionales
• Ciclos
• Código limpio
POO
• Prototipos
• Clases
• Organización
• Encapsulación, herencia y composición
Node.js
• Entorno
• Asincronía
• npm
• WebSockets
• Event loop
• Instalación
• Seguridad
Selección de herramientas
• Mantenibilidad
• Comunidad
• Rendimiento
• Licencia
• Matriz de decisión

Glosario

Alcance (scope): zona del programa donde una variable es visible y accesible.

Callback: función que se ejecuta como respuesta a un evento o al finalizar una operación asíncrona.

Closure: función que conserva acceso a variables de su contexto léxico incluso después de que ese contexto terminó.

Composición: enfoque de diseño donde un objeto se construye a partir de otros componentes ("tiene un") en lugar de heredar ("es un").

Concurrencia: capacidad de manejar múltiples tareas o conexiones en un intervalo de tiempo, sin necesariamente ejecutarlas todas en paralelo.

Encapsulación: principio de la Programación Orientada a Objetos (POO) que oculta el estado interno de un objeto y expone operaciones controladas para mantener su integridad.

Event loop: mecanismo de ejecución que gestiona la ejecución de `callbacks` a través de fases, coordinando eventos de temporizadores y operaciones de Entrada/Salida (I/O) sin bloqueo.

I/O (Entrada/Salida): operaciones de red, archivos o base de datos que implican esperar respuesta externa.

Inmutabilidad: estrategia donde los datos no se modifican directamente; se crean nuevas versiones del objeto, reduciendo errores por mutación accidental.

Mutabilidad: capacidad de un objeto de cambiar su estado interno después de haber sido creado.

NPM: gestor de Paquetes de Node (`Node Package Manager`), utilizado para instalar dependencias y gestionar versiones en proyectos JavaScript.

Prototipo: mecanismo base de herencia en JavaScript; los objetos pueden delegar la búsqueda de propiedades a su prototipo cuando no las encuentran directamente.

Socket: extremo de comunicación que representa una conexión entre cliente y servidor, usualmente sobre el Protocolo de Control de Transmisión (TCP).

TCP: protocolo de Control de Transmisión (Transmission Control Protocol). Protocolo orientado a conexión que garantiza entrega confiable y en orden de datos entre dos extremos.

Referencias bibliográficas

Armstrong, J. (2019). Programming Erlang: Software for a concurrent world (2.ª ed.). Pragmatic Bookshelf.

Crockford, D. (2008). JavaScript: The good parts. O'Reilly Media.

Haverbeke, M. (2020). Eloquent JavaScript: A modern introduction to programming (3.ª ed.). No Starch Press.

Mozilla. (s. f.). Arrow function expressions. MDN Web Docs.

Node.js Project. (s. f.). Don't block the event loop (or the worker pool). Node.js Documentation.

Simpson, K. (2015). You don't know JS: Scope & closures. O'Reilly Media.

Simpson, K. (2015). You don't know JS: This & object prototypes. O'Reilly Media.

Socket.IO. (s. f.). Redis adapter. Socket.IO Documentation.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Edison Eduardo Mantilla Cuadros	Responsable de línea de producción	Centro Agroturístico - Regional Santander
Carlos Andrés Bonza Reyes	Experto temático	Centro Agroturístico - Regional Santander
Angélica Varón Quintero	Evaluadora instruccional	Centro Agroturístico - Regional Santander
Julián Fernando Vanegas Vega	Diseñador de contenidos	Centro Agroturístico - Regional Santander
Andrea Paola Botello De la Rosa	Desarrolladora full stack	Centro Agroturístico - Regional Santander
María Alejandra Vera Briceño	Animadora y productora audiovisual	Centro Agroturístico - Regional Santander
Laura Paola Gelvéz Manosalva	Validadora y vinculadora de recursos educativos digitales	Centro Agroturístico - Regional Santander
Sandra Liliana Cristancho Cruz	Evaluadora de contenidos inclusivos y accesibles	Centro Agroturístico - Regional Santander